

Sistema de Recomendación de Productos para E-commerce

1. Contexto Empresarial

Empresa: ShopFast, tienda online de electrónica y tecnología con 50,000 clientes activos y catálogo de 5,000 productos.

Problemática: Las ventas se han estancado en los últimos 6 meses. Los clientes tienen dificultad encontrando productos relevantes. La tasa de conversión es solo 2.3% y el valor promedio de compra es \$85 USD.

Objetivo del Proyecto: Implementar un sistema de recomendación de productos que aumente la tasa de conversión a 4% y el ticket promedio a \$120 USD mediante sugerencias personalizadas.

Requerimientos Clave:

- Generar recomendaciones en menos de 500ms
- Procesar historial de 50,000 usuarios
- Disponibilidad 99% en horario comercial
- Presupuesto AWS: máximo \$200/mes

2. Componentes de la Arquitectura de IA

2.1 Capa de Datos

- **Fuentes:** Base de datos MySQL con historial de compras y clics, S3 con 12 meses de datos históricos (2GB)
- **Features:** user_id, product_id, interaction_type, rating, timestamp, categoría, precio
- **Procesamiento:** Scripts Python con pandas para limpieza y transformación
- **Función:** Proporcionar datos limpios para entrenar el modelo

2.2 Capa de Modelo

- **Tipo:** Filtrado colaborativo con Matrix Factorization (SVD)
- **Framework:** Python Surprise 1.1.3
- **Training:** 150,000 interacciones, 50 factores latentes, 20 épocas
- **Métricas:** RMSE 0.85, Precisión@10 = 0.72
- **Output:** Top 10 productos por usuario con score de relevancia
- **Función:** Predecir productos de interés basado en comportamiento similar

2.3 Capa de API

- **Tecnología:** Flask en AWS Lambda + API Gateway
- **Endpoints:**
 - GET /recommendations/{user_id} - Retorna 10 productos recomendados
 - POST /track-interaction - Registra interacciones
 - GET /health - Estado del servicio
- **Seguridad:** API Key, rate limiting 100 req/min, HTTPS
- **Función:** Exponer modelo como servicio REST

2.4 Capa de Interfaz

- **Tecnología:** Componentes React
- **Ubicaciones:**
 - Homepage: "Recomendado para ti" (10 productos)
 - Página producto: "También te puede interesar" (5 productos)
- **Elementos:** Imagen, nombre, precio, botón "Ver producto"
- **Función:** Presentar recomendaciones al usuario final

2.5 Componentes No Funcionales

- **Observabilidad:** CloudWatch Logs/Metrics, dashboard con latencia y errores, alertas automáticas
- **Seguridad:** HTTPS, API Keys en Parameter Store, IAM roles mínimo privilegio, datos anonimizados
- **Confiabilidad:** Retry automático, versionado de modelos, fallback a productos más vendidos

3. Infraestructura Cloud

3.1 Recursos Computacionales

Entrenamiento:

- Instancia: ml.t3.medium (2 vCPU, 4GB RAM)
- Justificación: SVD es CPU-intensivo, no requiere GPU, 150K registros entrenan en 15 minutos
- Frecuencia: Semanal, domingos 2 AM
- Costo: \$0.20/mes

Inferencia:

- Servicio: AWS Lambda (1GB memoria)
- Justificación: Serverless elimina costos sin tráfico, escala automáticamente
- Performance: Cold start 2s, warm requests 300ms
- Costo: \$28/mes (60,000 invocaciones)

Almacenamiento:

- S3: 2GB datos/modelos (\$0.05/mes)

- RDS MySQL db.t3.micro: 20GB (\$16/mes)

Costo Total: \$50/mes vs \$180/mes con EC2 24/7

3.2 Servicios AWS

SageMaker: Training jobs semanales automatizados

Lambda + API Gateway: Inferencia serverless con auto-scaling 0-1000 instancias

S3: Almacenamiento datos y modelos

CloudWatch: Logs, métricas y alertas

EventBridge: Trigger reentrenamiento semanal

Justificación: Arquitectura serverless óptima para proyectos pequeños-medianos, pago por uso, cero mantenimiento de servidores, escalabilidad automática.

3.3 Principios de Diseño

Escalabilidad:

- Lambda auto-scaling 0-1000 instancias
- Validación: 500 usuarios concurrentes, 0 errores, latencia p95 420ms

Flexibilidad:

- Modelo como archivo independiente, fácil cambiar algoritmo
- Configuración externalizada
- Validación: Cambio SVD→ALS en 2 horas

Seguridad:

- API Key obligatoria, HTTPS por defecto, logs sin PII
- Validación: Penetration testing sin vulnerabilidades críticas

Observabilidad:

- Logs estructurados con request_id, dashboard tiempo real
- Validación: Detección de problemas en <15 minutos

Confiabilidad:

- Retry automático, versionado modelos, fallback a top vendidos
- Validación: 99.4% uptime, fallback activa en <500ms

4. Integración y Despliegue

4.1 CI/CD Pipeline

Herramientas: GitHub Actions + AWS SAM + pytest

Proceso:

1. Push a main → Build (tests, linting, security scan)
2. Deploy staging → Tests integración (5 validaciones)
3. Aprobación manual
4. Deploy producción → Canary 10% cada minuto
5. Rollback automático si error rate >3%

Frecuencia: Modelo semanal automático, código cada 2-3 semanas

4.2 Integración Frontend

Flujo:

- React app llama API con user_id
- API Gateway → Lambda → Genera predicciones
- Retorna JSON con 10 productos
- React renderiza cards visuales

Manejo errores:

- Timeout 2s → mostrar productos más vendidos
- Error 5xx → retry 1 vez, luego fallback
- Fallback: productos más vendidos cacheados

4.3 Monitoreo

Métricas Técnicas:

- Requests/hora: 2,100 pico
- Latencia: p50 235ms, p95 420ms, p99 780ms
- Error rate: 0.8%
- Cold starts: 4.2%

Métricas Negocio:

- Conversión: 2.3% → 3.7% (+61%)
- Ticket promedio: \$85 → \$108 (+27%)
- Click-through recomendaciones: 14.2%
- Productos recomendados en carrito: 31%

Alertas:

- Crítica: Error rate >5%, latencia p95 >1.5s
- Warning: Error rate 3-5%, latencia 800ms-1.5s
- Reporte semanal: Uptime, costos, performance

5. Resultados e Impacto

ROI: 340% después de 3 meses (inversión \$6,500, retorno \$22,100)

Valor generado: \$18,400 adicionales por trimestre

Satisfacción: NPS aumentó de 42 a 58

Conclusión: Sistema simple, efectivo y escalable que demuestra valor empresarial real con arquitectura cloud optimizada para costos y rendimiento.